

```
quit ->
    io:format("Pong got quit-n",[]);
{X , Ping_PID} ->
    io:format("Pong received ~w-n",[X]),
    Ping_PID ! {X,pong},
    pong()
end.
ping_pong() ->
    Pong_PID = spawn(cp, pong, []),
    spawn(cp,ping, [3, Pong_PID,ping]).
```

Note how easy it is to send a message – Process ID ! <data>. Data can be an atom, a tuple, a list, etc. The processing of a ‘receive’ statement is very much like a ‘case’ statement. Depending upon what is received, the appropriate code is executed.

The *ping_pong* function spawns both processes. Pong is started first and its process ID is passed to ping. Once ping starts, it sends a message to pong along with its process ID.

Now, from the Erlang shell...

```
8> c(cp).
9> cp:ping_pong().
Pong received ping
Ping received [ping,pong]
...
```

You can now add some complication and have multiple processes pinging pong. So, in *ping_pong*, *spawn ping* twice as follows:

```
spawn(cp,ping, [3, Pong_PID,ping]),
spawn(cp,ping, [3, Pong_PID,ping_ping]).
```

You should alter *pong* so that it does not die after receiving a quit message, as follows:

```
quit ->
    io:format("Pong got quit-n",[]),
    pong();
```

Now, compile *cp* and run *ping_pong* again. You can notice that you have the structure of a client/server program.

However, if the *ping* processes need to be run at various occasions, it may not be possible to have the process ID of the *pong* process. Erlang provides an option to register a process. Other processes can then use the symbolic reference. You can alter the *ping_pong* function as follows:

```
ping_pong() ->
    register(pong, spawn(cp, pong, [])),
    spawn(cp, ping, [3, ping]),
    spawn(cp, ping, [3, ping_ping]).
```

Notice that you no longer pass the process ID of *pong* to *ping*. *Ping* will use the symbolic name *pong* to send a message. The changes to the export statement and the *ping* function will be as follows:

```
-export([n_echo/2, ping/2, pong/0, ping_pong/0]).
ping(0, Ping) ->
    pong ! Quit,
    io:format("~w finished-n", [Ping]);
ping(N, Ping) ->
    pong ! {Ping, self()},
    receive
        X ->
            io:format("~w received ~w-n",[Ping,X])
    end,
    ping(N-1, Ping).
```

With this brief exploration, you should be convinced that Erlang is an excellent option for writing code to exploit the multiple cores of the current processors. So try it out. Learning a new language is fun! 

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com>, <http://sethanil.blogspot.com>, and reach him via email at anil@sethanil.com

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite Magazine on
Open Source is now on the Web, too.

LinuxForU.com

Follow us on Twitter@LinuxForU